

ЭКЗАМЕНАЦИОННЫЙ БИЛЕТ (ТРЕНИРОВОЧНЫЙ)

ЗАДАНИЕ:

Используя навыки, полученные в процессе обучения, на основании полученной технической документации (см. Приложение), реализуйте бэкенд веб-приложения для управления данными согласно вашему варианту.

ТРЕБОВАНИЯ К ПРИЛОЖЕНИЮ:

- Реализовать модель данных (таблицу) в соответствии с вариантом.
- Выполнить миграции.
- Реализовать веб-интерфейс (админ-панель Django, SWAGGER или собственную реализацию) для выполнения CRUD-операций (создание, чтение, редактирование, удаление записей).
- Реализовать валидацию данных на сервере согласно правилам варианта. При ошибках валидации пользователь должен получать сообщение.
- Обращать ошибки: при попытке доступа к несуществующей записи – HTTP 404; при неверных данных – возврат на форму с сообщением (HTTP 400).
- Реализовать middleware, который подсчитывает:
 - общее количество запросов;
 - количество запросов с ответом 2xx;
 - количество запросов с ответом 4xx;
 - количество запросов с ответом 5xx.Выводить накопленную статистику в консоль или в файл `metrics.log`.
- Вынести настройки (DEBUG, SECRET_KEY, параметры БД) в переменные окружения (файл `.env`).
- Написать интеграционный тест (через `django.test` или `pytest`), проверяющий, что главная страница (или специальный эндпоинт `/ping/`) доступна и возвращает HTTP 200.
- Настроить корректную отдачу статических файлов при `DEBUG=False` (админ-панель должна отображаться со стилями).
- По желанию (дополнительные баллы) можно реализовать собственный веб-интерфейс на основе Bootstrap (список, формы, подтверждение удаления).

ТЕХНОЛОГИЧЕСКАЯ ПОСЛЕДОВАТЕЛЬНОСТЬ ВЫПОЛНЕНИЯ:

- 1. Подготовительный этап – изучить Приложение, определить последовательность действий.
- 2. Основной этап – создать проект, модель, миграции, представления, middleware, конфигурацию, тест.
- 3. Контрольно-сдаточный этап – проверить работоспособность, предъявить комиссии, ответить на вопросы.
- 4. Заключительный этап – привести в порядок рабочее место.

**Приложение к экзаменационному билету
(тренировочные варианты)**

Вариант 1. Товары (products)

Создайте таблицу `products` со следующей структурой:

Поле	Тип данных	Описание
id	INTEGER (PK)	Уникальный идентификатор товара
name	VARCHAR(150)	Название товара, не пустое
category	VARCHAR(100)	Категория товара
price	DECIMAL(10,2)	Цена, больше 0
sku	VARCHAR(50)	Артикул, уникальный
created_at	TIMESTAMP	Дата создания (автоматически)

Валидация:

- `name` – не может быть пустым.
- `price` – должно быть > 0 .
- `sku` – должно быть уникальным (при попытке создать товар с уже существующим `sku` – ошибка).

Обработка ошибок:

- Товар с указанным `id` не найден – 404.
- Невалидные данные (пустое имя, цена ≤ 0 , дубликат `sku`) – возврат на форму с сообщением.

Вариант 2. Мероприятия (events)

Создайте таблицу `events` со следующей структурой:

Поле	Тип данных	Описание
<code>id</code>	INTEGER (PK)	Уникальный идентификатор мероприятия
<code>title</code>	VARCHAR(200)	Название, не пустое
<code>location</code>	VARCHAR(150)	Место проведения
<code>event_date</code>	TIMESTAMP	Дата и время проведения (только будущее)
<code>max_guests</code>	INTEGER	Максимальное количество гостей, > 0
<code>created_at</code>	TIMESTAMP	Дата создания (автоматически)

Валидация:

- `title` – не может быть пустым.

- `event_date` – должна быть в будущем (нельзя создать мероприятие в прошлом).
- `max_guests` – целое положительное число (>0).

Обработка ошибок:

- Мероприятие с указанным `id` не найдено – 404.
- Невалидные данные (пустое название, дата в прошлом, $\text{max_guests} \leq 0$) – возврат на форму с сообщением.

Оценочный лист (базовая часть – 80 баллов)

№	Критерий	Баллы	Как проверить
1	Модель создана, миграции выполнены	10	Посмотреть файл <code>models.py</code> , выполнить <code>showmigrations</code>
2	CRUD через админ-панель (создание, редактирование, удаление, просмотр списка)	20	Зайти в админку, выполнить операции
3	Валидация данных	10	Попробовать сохранить невалидный объект через админку – должна быть ошибка

4	Обработка 404	5	Перейти в админке на несуществующий ID – страница 404
5	Middleware для метрик (общее число запросов, 2xx, 4xx, 5xx)	15	Открыть несколько страниц, проверить вывод в консоль или файл
6	Конфигурация через <code>.env</code> (DEBUG=False, SECRET_KEY)	5	Проверить файл <code>.env</code> и чтение в settings.py
7	Интеграционный тест (GET /ping/ → 200)	5	Запустить <code>python manage.py test</code>
8	Статика при <code>DEBUG=False</code> (админка со стилями)	10	Установить <code>DEBUG=False</code> , перезапустить сервер, открыть админку – есть CSS
ИТОГО		80	

Дополнительная часть (фронтенд) – до 20 баллов

№	Критерий	Баллы
9	Реализован веб-интерфейс со списком, формами, подтверждением удаления	15

10

Интерфейс адаптирован под конкретную модель (правильные названия полей, ссылки)

5

Итого максимум: 100 баллов

Шкала перевода в 5-балльную оценку:

- 80–100 – «отлично»
- 60–79 – «хорошо»
- 40–59 – «удовлетворительно»
- менее 40 – «неудовлетворительно»

Базовые шаблоны Bootstrap

Для выполнения дополнительной части (собственный веб-интерфейс) студенты могут использовать следующие готовые шаблоны. Их нужно скопировать в папку `templates` своего приложения и адаптировать под свою модель (названия полей, URL-маршруты).

Файл 1: `base.html` (основной каркас)

```
html
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}Менеджер данных{% endblock %}</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.c
ss" rel="stylesheet">
</head>
<body>
  <nav class="navbar navbar-expand-lg bg-body-tertiary">
    <div class="container">
```

```

        <a class="navbar-brand" href="{% url 'index' %}">Менеджер
данных</a>
    </div>
</nav>
<div class="container mt-4">
    {% block content %}{% endblock %}
</div>
</body>
</html>

```

Файл 2: `index.html` (страница со списком записей)

```

html
{% extends 'base.html' %}

{% block title %}Список записей{% endblock %}

{% block content %}
<h1>Список записей</h1>
<a href="{% url 'create' %}" class="btn btn-primary mb-3">Добавить
запись</a>

<table class="table table-striped">
    <thead>
        <tr>
            <th>ID</th>
            <th>Наименование</th>
            <th>Дополнительные поля</th>
            <th>Действия</th>
        </tr>
    </thead>
    <tbody>
        {% for obj in object_list %}
        <tr>
            <td>{{ obj.id }}</td>
            <td>{{ obj.name }}</td>  <!-- Поменять на нужное поле (например,
title) -->
            <td>
                <!-- Здесь можно вывести другие поля, например, price или
location -->
            </td>

```

```

        <td>
            <a href="{% url 'update' obj.id %}" class="btn btn-sm
btn-warning">Изменить</a>
            <a href="{% url 'delete' obj.id %}" class="btn btn-sm
btn-danger">Удалить</a>
        </td>
    </tr>
    {% empty %}
    <tr>
        <td colspan="4">Нет записей</td>
    </tr>
    {% endfor %}
</tbody>
</table>

{% endblock %}

```

Примечание для студента: замените `obj.name` на название поля из вашей модели (например, `obj.title` для мероприятий или `obj.full_name` для клиентов). Дополнительные поля можно добавить в третью колонку.

Файл 3: `form.html` (создание / редактирование записи)

```

html

{% extends 'base.html' %}

{% block title %}{% if object %}Редактирование{% else %}Создание{% endif
%}{% endblock %}

{% block content %}
<h1>{% if object %}Редактирование записи{% else %}Новая запись{% endif
%}</h1>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit" class="btn btn-success">Сохранить</button>
    <a href="{% url 'index' %}" class="btn btn-secondary">Отмена</a>
</form>

{% endblock %}

```

Файл 4: `confirm_delete.html` (подтверждение удаления)

```
html

{% extends 'base.html' %}

{% block title %}Удаление записи{% endblock %}

{% block content %}
<h1>Удалить запись?</h1>
<p>Вы уверены, что хотите удалить "{{ object }}"?</p>
<form method="post">
    {% csrf_token %}
    <button type="submit" class="btn btn-danger">Да, удалить</button>
    <a href="{% url 'index' %}" class="btn btn-secondary">Нет, отмена</a>
</form>

{% endblock %}
```

Инструкция по использованию шаблонов

1. В вашем приложении (например, `products`) создайте папку `templates/products/`.
2. Скопируйте приведённые выше файлы в эту папку.
3. Убедитесь, что в файле `urls.py` заданы правильные имена маршрутов (`index`, `create`, `update`, `delete`). Пример:

```
python

urlpatterns = [
    path('', views.list_view, name='index'),
    path('create/', views.create_view, name='create'),
    path('<int:pk>/update/', views.update_view, name='update'),
    path('<int:pk>/delete/', views.delete_view, name='delete'),
]
```

4. В представлениях (`views.py`) необходимо передавать в контекст переменные с именами, ожидаемыми шаблонами: `object_list` для списка, `form` и `object` для формы, `object` для подтверждения удаления.
5. При необходимости измените названия полей в таблице (например, `obj.name` на `obj.title`), чтобы они соответствовали вашей модели.